DevOps High-Availability Stack

A Complete Docker-Based Infrastructure Project

Step-by-Step Guide · Architecture Overview · Component Reference

Table of Contents

1. Project Overview

This project provisions a **complete high-availability DevOps infrastructure** using Docker Compose. It simulates a real-world production environment with:

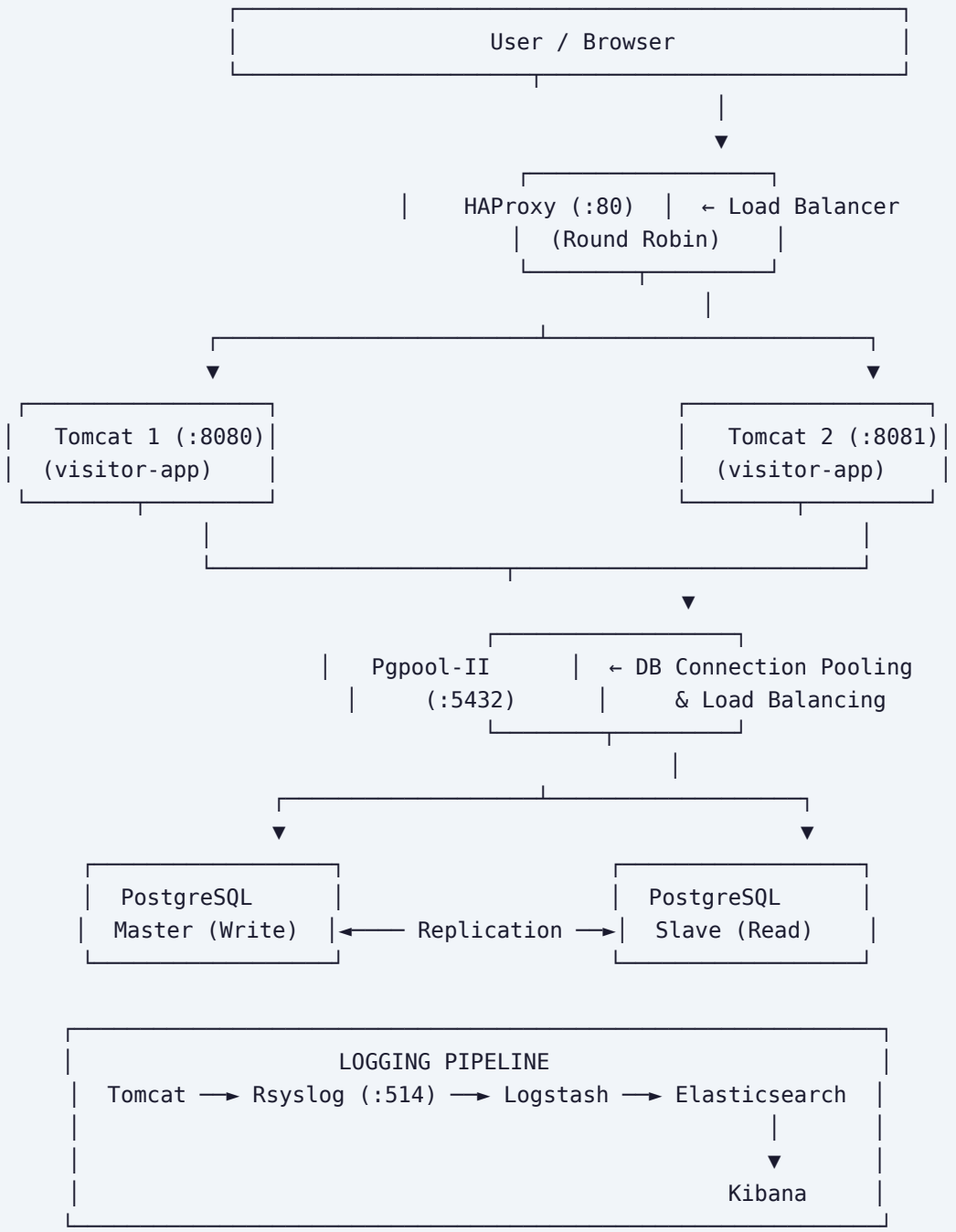
- **Load balancing** across multiple application servers
- **Database replication** with read/write splitting
- **Centralized logging** pipeline (ELK Stack)
- **Infrastructure monitoring** (Prometheus + Grafana)

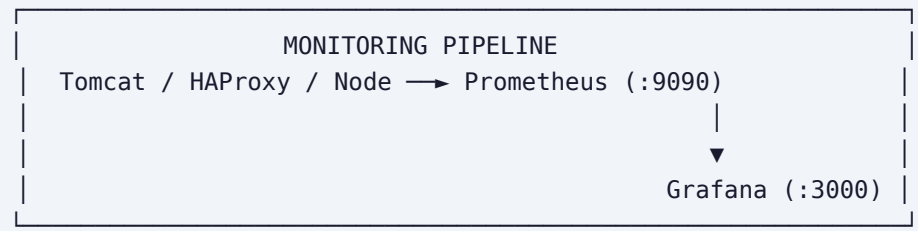
The entire stack runs in Docker containers orchestrated by a single `docker-compose.yml` file. This makes it easy to learn, reproduce, and extend.

What You Will Learn

- How HAProxy distributes traffic across Tomcat servers
- How to set up PostgreSQL streaming replication (master-slave)
- How Pgpool-II provides connection pooling and load balancing
- How rsyslog collects logs and forwards them to Logstash
- How Elasticsearch indexes logs and Kibana visualizes them
- How Prometheus scrapes metrics and Grafana displays dashboards

2. Architecture Diagram




MONITORING PIPELINE
Tomcat / HAProxy / Node
Prometheus (:9090)
Grafana (:3000)

3. Prerequisites

Before starting, ensure you have the following installed:

Required Software

- **Docker Engine** (version 20.10+) and **Docker Compose** (v2)
- **Java 11+** (only if building the app locally)
- **Apache Maven** (only if building the app locally)
- A terminal / command line interface

Check Docker:

```
docker --version && docker compose version
```

System Requirements

- **CPU:** 2+ cores
- **RAM:** 6+ GB (the ELK stack is memory-hungry)
- **Disk:** 10+ GB free

4. Project Structure

```
Devops-project/
├── docker-compose.yml          # Orchestrates all 14 containers
├── prometheus.yml              # Prometheus scrape config
├── logstash.conf                # Logstash pipeline config
├── haproxy/
│   └── haproxy.cfg             # HAProxy load balancer config
├── pgpool/
│   ├── Dockerfile              # Custom Pgpool image build
│   ├── pgpool.conf             # Pgpool configuration
│   ├── pcp.conf                # Pgpool admin auth
│   └── pool_passwd             # Pgpool password file
├── rsyslog-config/
│   └── rsyslog.conf            # Rsyslog receiver config
├── test-app/
│   ├── pom.xml                 # Maven project definition
│   ├── index.jsp               # (optional) JSP fallback
│   ├── src/main/java/com/
│   │   ├── DBUtil.java         # PostgreSQL connection utility
│   │   ├── HelloServlet.java   # Main dashboard servlet
│   │   ├── MetricsServlet.java # Prometheus metrics endpoint
│   │   └── AppMetrics.java     # In-memory metrics counters
│   └── target/
│       └── visitor-app/        # Pre-built WAR (deployed to Tomcat)
├── logs/                       # Shared log volume
└── devops-summary.pdf          # (existing summary)
```

5. Step 1: Load Balancer (HAProxy)

What It Does

HAProxy sits at the front of the application stack. It receives all incoming HTTP traffic on port 80 (mapped to host port **8082**) and distributes requests across two Tomcat servers using **round-robin** load balancing.

Configuration (`haproxy/haproxy.cfg`)

```
frontend http_front
    bind *:80
    default_backend tomcat_back

backend tomcat_back
    balance roundrobin
    option httpchk GET /
    server tomcat1 tomcat1:8080 check
    server tomcat2 tomcat2:8080 check

frontend stats
    bind *:8404
    http-request use-service prometheus-exporter if { path /metrics }
    stats enable
    stats uri /stats
    stats auth admin:admin
```

Key Concepts

- **Frontend:** The entry point that listens for connections
- **Backend:** The pool of servers to forward traffic to
- **Health checks:** HAProxy pings `GET /` on each server every few seconds. If a server fails, traffic is redirected to the healthy one

- **Stats:** Built-in monitoring page at `/stats` and Prometheus metrics at `/metrics` (port 8404)

> **Try it:** Open http://localhost:8404/stats (login: admin / admin) to see live traffic distribution.

6. Step 2: Application Stack (Tomcat + Java App)

What It Does

Two Tomcat 9 containers run a Java servlet application called **visitor-app**. Each request:

1. Connects to PostgreSQL via Pgpool
2. Creates a `visits` table if it doesn't exist
3. Inserts a new row (each visit = one row)
4. Queries the total count
5. Renders a styled HTML dashboard showing:
   - Server hostname (to identify which Tomcat handled the request)
   - Total visitor count
   - Response time
   - Database status (Online / Error)

Application Source Code

HelloServlet.java - The main dashboard servlet mapped to `/`:

```
@WebServlet("/")
public class HelloServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse resp) {
        // 1. Connect to DB via DBUtil
        // 2. CREATE TABLE IF NOT EXISTS visits
        // 3. INSERT INTO visits DEFAULT VALUES
        // 4. SELECT COUNT(*) FROM visits
        // 5. Render HTML dashboard with hostname, count, response time, DB status
    }
}
```

MetricsServlet.java - Prometheus metrics endpoint at `/metrics`:

```
# HELP app_requests_total Total number of requests
# TYPE app_requests_total counter
app_requests_total{instance="tomcat1"} 42

# HELP app_db_errors_total Total database errors
# TYPE app_db_errors_total counter
app_db_errors_total{instance="tomcat1"} 0

# HELP app_response_time_seconds_avg Average response time
# TYPE app_response_time_seconds_avg gauge
app_response_time_seconds_avg{instance="tomcat1"} 0.015
```

AppMetrics.java - In-memory counters using AtomicLong

DBUtil.java - JDBC connection utility pointing to `pgpool:5432/mydb`

> **Why two Tomcat instances?** This demonstrates high availability. If one fails, HAProxy routes all traffic to the other. In production, you'd run many more behind the load balancer.

7. Step 3: Database Layer (PostgreSQL Master-Slave + Pgpool)

PostgreSQL Master-Slave Replication

The database layer uses **streaming replication**:

- **pg-master** handles all write operations
- **pg-slave** receives real-time replication from the master and can serve read queries

Pgpool-II

Pgpool sits between the application and the database. It provides:

- **Connection pooling** - Reuses database connections to reduce overhead
- **Load balancing** - Distributes read queries across master and slave
- **Automatic failover** - Detects down nodes and routes traffic away

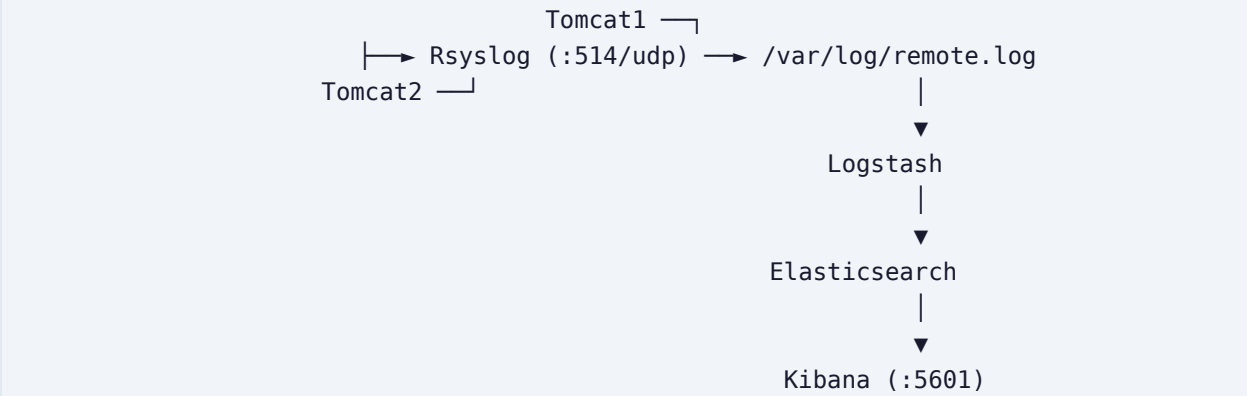
Key environment variables in `docker-compose.yml`:

```
PGPOOL_BACKEND_NODES=0:pg-master:5432,1:pg-slave:5432
PGPOOL_SR_CHECK_USER=postgres
PGPOOL_SR_CHECK_PASSWORD=mysecretpassword
PGPOOL_ENABLE_LOAD_BALANCING=yes
```

> **Connect to Pgpool:**
>
> ```
> docker exec -it pgpool psql -U postgres -d mydb -c "SELECT * FROM visits;"
> ```

8. Step 4: Centralized Logging (Rsyslog + ELK Stack)

Data Flow



Component Details

Component	Role	Port
Rsyslog	Receives syslog messages from Tomcat containers over UDP/TCP port 514, writes to `/var/log/remote.log`	514
Logstash	Watches the log file, parses lines using GROK patterns, sends structured JSON to Elasticsearch	-
Elasticsearch	Indexes and stores log data for fast search	9200
Kibana	Web UI for searching and visualizing logs	5601

Rsyslog Config (`rsyslog-config/rsyslog.conf`)

```
module(load="imudp")
input(type="imudp" port="514")
module(load="imtcp")
input(type="imtcp" port="514")

*.* action(type="omfile" file="/var/log/remote.log")
*.* action(type="omfile" file="/dev/stdout")
```

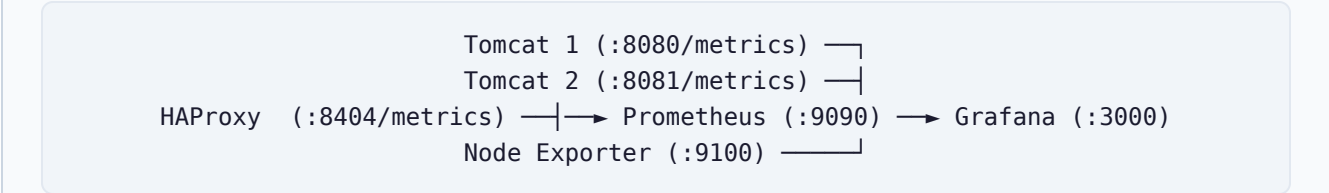
Logstash Config (`logstash.conf`)

```
input {
  file {
    path => "/var/log/host-logs/*.log"
    start_position => "beginning"
  }
}

filter {
  grok {
    match => { "message" => "%{SYSLOGTIMESTAMP} %{SYSLOGHOST} %{DATA}..." }
  }
}

output {
  elasticsearch { hosts => ["http://elasticsearch:9200"] }
}
```

> **Kibana Setup:** Go to http://localhost:5601, navigate to Stack Management → Index Patterns, create pattern `syslog-*`, then explore logs in the Discover tab.

9. Step 5: Monitoring (Prometheus + Grafana)

Data Flow



Prometheus Config (`prometheus.yml`)

```
scrape_configs:
  - job_name: 'test-app'
    metrics_path: '/metrics'
    static_configs:
      - targets: ['tomcat1:8080', 'tomcat2:8080']

  - job_name: 'node-exporter'
    static_configs:
      - targets: ['node-exporter:9100']

  - job_name: 'haproxy'
    metrics_path: '/metrics'
    static_configs:
      - targets: ['haproxy:8404']
```

What Gets Monitored

Metric	Source	Description
`app_requests_total`	Tomcat	Request count per instance
`app_db_errors_total`	Tomcat	Database error count per instance
`app_response_time_seconds_avg`	Tomcat	Average response time
`node_cpu_seconds_total`	Node Exporter	Host CPU usage
`node_memory_MemAvailable_bytes`	Node Exporter	Host available memory
`haproxy_server_http_responses_total`	HAProxy	HTTP response codes per server

Explore metrics:

```
# Prometheus expression browser
http://localhost:9090

# Try querying:
app_requests_total
rate(app_requests_total[1m])
```

Grafana

Grafana connects to Prometheus as a data source and provides dashboards. Default login: **admin / admin**.

Quick dashboard: In Grafana, go to Configuration → Data Sources → Add Prometheus (URL: http://prometheus:9090), then create a new dashboard with panels for request count, response time, and DB errors.

10. Step 6: Putting It All Together (Docker Compose)

Starting the Stack

```
cd Devops-project
docker compose up -d
```

This pulls all images and starts all 14 containers in the correct order (respecting `depends_on`).

Checking Status

```
docker compose ps
```

All containers should show `Up` status.

Viewing Logs

```
# All services
docker compose logs -f

# Specific service
docker compose logs -f tomcat1
docker compose logs -f pgpool
```

Stopping the Stack

```
docker compose down
```

Rebuilding After Changes

```
# If you modify the Java app:
cd test-app
mvn clean package

# Then restart the Tomcat containers:
docker compose restart tomcat1 tomcat2
```

11. Accessing Services

Service	URL	Credentials
Application (via HAProxy)	http://localhost:8082	-
Tomcat 1 (direct)	http://localhost:8080	-
Tomcat 2 (direct)	http://localhost:8081	-
HAProxy Stats	http://localhost:8404/stats	admin / admin
Pgpool (Postgres)	`localhost:5432`	postgres / mysecretpassword
Elasticsearch	http://localhost:9200	-
Kibana	http://localhost:5601	-
Prometheus	http://localhost:9090	-
Grafana	http://localhost:3000	admin / admin

12. Scaling & Production Considerations

What's Missing for Production

- **SSL/TLS:** Add a reverse proxy (Nginx/Traefik) with Let's Encrypt for HTTPS
- **Persistent config for Pgpool setup:** pg-master and pg-slave are not auto-configured with replication scripts; a production setup would include initialization SQL
- **Docker secrets:** Passwords are in plaintext in docker-compose.yml
- **Volume backups:** No backup strategy for PostgreSQL or Elasticsearch data
- **Orchestration:** For multi-node setups, use Kubernetes or Docker Swarm
- **Health checks:** Docker HEALTHCHECK directives would improve reliability
- **Resource limits:** Add `mem_limit` and `cpus` constraints

Ideas to Extend This Project

- Add a CI/CD pipeline (GitHub Actions / Jenkins) to build and deploy the app
- Add alerting rules in Prometheus + Alertmanager
- Add automated DB failover testing (stop pg-master, verify pgpool routes to slave)
- Replace simple file-based logging with structured JSON logging
- Add a message queue (Kafka / RabbitMQ) between Tomcat and Logstash

End of Guide

DevOps High-Availability Stack — Docker Compose Project